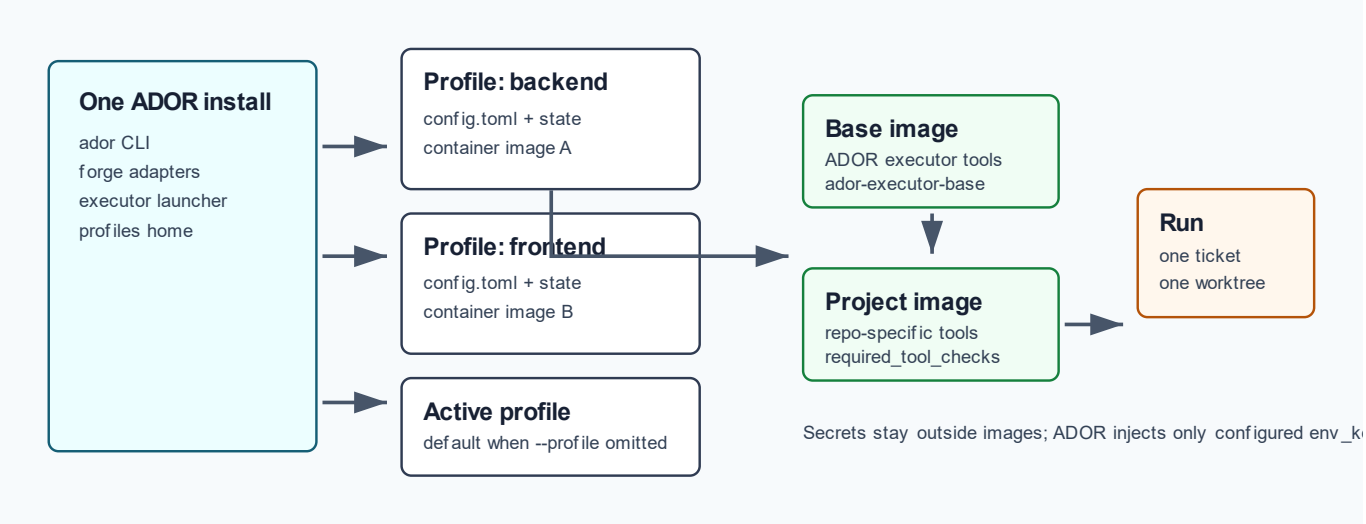
Profiles And Container Images

Applies to ADOR 0.5.1.

Use this when you operate more than one runtime project, switch active projects, or need to decide which executor image a profile should use.



Mental Model

One ADOR install can serve many runtime projects. Each runtime project should have its own profile with its own `config.toml`, metadata, state, metrics, and container image selection.

Config resolution order:

```
-c FILE > --profile NAME > active profile > ./config.toml
```

Profile Commands

```
.\.venv\Scripts\ador.exe profile list
.\.venv\Scripts\ador.exe profile show <profile-name>
.\.venv\Scripts\ador.exe profile use <profile-name>
.\.venv\Scripts\ador.exe profile path <profile-name>
```

```
./.venv/bin/ador profile list
./.venv/bin/ador profile show <profile-name>
./.venv/bin/ador profile use <profile-name>
./.venv/bin/ador profile path <profile-name>
```

Use explicit `--profile` for scripts and first validation. Use an active profile for daily interactive work after you trust the selection.

Container Image Layers

Layer	What belongs there	Example
ADOR executor base	Shared executor tooling such as Python, Node, git, uv, and executor CLIs.	`ador-executor-base:0.2.0`
Runtime-project image	Project-specific gate tools and caches.	`ador-executor-cockpit-gcp:latest` or a project tag
Per-run container	One isolated claim of one ticket.	Created and removed by ADOR.

Do not put secrets into images. Model provider keys and forge tokens belong in the host environment or `.env`, and only configured `env_keys` reach the executor container.

Select The Image In The Profile

The profile's `[container] image` chooses the runtime image:

```
[container]
runtime = "docker"
image = "ador-executor-cockpit-gcp:latest"
required_tool_checks = [
  "make --version",
  "shellcheck --version",
  "terraform --version",
  "tflint --version",
]
```

For rootless Podman, include the verified user namespace run argument:

```
[container]
runtime = "podman"
run_args = ["--userns=keep-id"]
```

Image Readiness Check

`required_tool_checks` is the fast failure point for missing project tools. If a project needs Java, Terraform, Make, ShellCheck, or another build dependency, put a simple version command here.

Good checks:

```
required_tool_checks = [
  "java -version",
  "mvn --version",
  "make --version",
]
```

Avoid checks that mutate the project, download dependencies, or require a live network. The point is to prove the selected image is usable before the executor spends a gate attempt.

Active Profile Safety

Before a real run, confirm the selected profile and store:

```
.\.venv\Scripts\ador.exe profile list
.\.venv\Scripts\ador.exe --profile <profile-name> status
```

```
./.venv/bin/ador profile list
./.venv/bin/ador --profile <profile-name> status
```

The `profile list` output marks the active profile with `*` and shows basic run statistics, platform, repo, runtime, last used date, and use count.